Binary tree implementation :

main.c

```c
#include <stdio.h>
#include <stdlib.h>

// TreeNode structure
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Create new node
struct TreeNode* createNode(int data) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof
        (struct TreeNode));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert (simple level-order insertion)
struct TreeNode* insert(struct TreeNode* root, int data) {
    if (root == NULL) {
        return createNode(data);
```

Output

```
Pre-order traversal: 10 20 40 50 30

=== Code Execution Successful ===
```

main.c

```c
    // Use queue for level-order insertion
    struct TreeNode* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        struct TreeNode* temp = queue[front++];

        if (temp->left == NULL) {
            temp->left = createNode(data);
            return root;
        } else {
            queue[rear++] = temp->left;
        }

        if (temp->right == NULL) {
            temp->right = createNode(data);
            return root;
        } else {
            queue[rear++] = temp->right;
        }
    }
```

Output

```
Pre-order traversal: 10 20 40 50 30

=== Code Execution Successful ===
```

main.c

```c
void pre_order_traversal(struct TreeNode* root) {
    if (root == NULL)
        return;

    printf("%d ", root->data);          // Visit root
    pre_order_traversal(root->left);    // Left
    pre_order_traversal(root->right);   // Right
}

// Main function
int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 10);
    insert(root, 20);
    insert(root, 30);
    insert(root, 40);
    insert(root, 50);

    printf("Pre-order traversal: ");
    pre_order_traversal(root);

    return 0;
```

Output

```
Pre-order traversal: 10 20 40 50 30

=== Code Execution Successful ===
```

Binary search tree implementation :

main.c

```c
#include <stdio.h>
#include <stdlib.h>

// TreeNode structure
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Create new node
struct TreeNode* createNode(int data) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof
        (struct TreeNode));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert into BST
struct TreeNode* insert(struct TreeNode* root, int key) {
    if (root == NULL)
        return createNode(key);
```

Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```

main.c

```c
    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);

    return root;
}

// Inorder Traversal (Left → Root → Right)
void in_order_traversal(struct TreeNode* root) {
    if (root == NULL)
        return;

    in_order_traversal(root->left);
    printf("%d ", root->data);
    in_order_traversal(root->right);
}

// Postorder Traversal (Left → Right → Root)
void post_order_traversal(struct TreeNode* root) {
    if (root == NULL)
        return;

```

Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```

main.c

```c
    printf("%d ", root->data);
}

// Main function
int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    printf("Inorder Traversal: ");
    in_order_traversal(root);

    printf("\nPostorder Traversal: ");
    post_order_traversal(root);

    return 0;
}
```

Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```